Let's forget difficult definitions. I will explain using one simple real-life example: building a college student registration app.

First, understand this: **Epic** and **User Story** are used to organize customer requirements so that the team can understand and build the software easily.

1 Imagine a customer comes to a software company

The customer says:

> "I want an app for college students. Students should be able to register, log in, view their marks, and update their profiles."

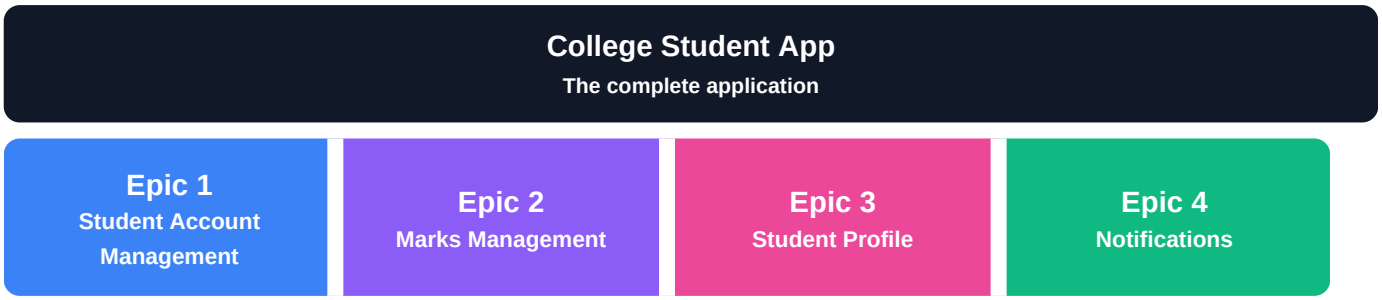
These are the customer's requirements. But there is a problem: there are many things to build.

How do we organize them? That's where Epics and User Stories help.

2 What is an Epic, and why do we create it?

Imagine you have a big school bag. Inside it, you keep separate books for different subjects.

Similarly, we group related software requirements under an Epic.



Why create Epics?

To group similar requirements together.

For example, registration and login both belong to Student Account Management.

Without Epics, all requirements may be mixed together, making it harder to organize and track the work.

3 What is a User Story, and why do we create it?

Now let's look inside Epic 1: Student Account Management.

The Epic name is big. It doesn't tell the developers exactly what the app must do.

So we divide it into smaller pieces of functionality called User Stories.

Epic: Student Account Management

> **User Story 1: Registration**
> As a new student, I want to register so that I can create an account.

> **User Story 2: Login**
> As a registered student, I want to log in so that I can access my account.

> **User Story 3: Password Reset**
> As a student, I want to reset my password so that I can regain access if I forget it.

Why not just keep the Epic? Why write User Stories?

Because the Epic is too broad to guide the team on one specific piece of work.

Suppose the developer sees only:

> “Student Account Management.”

What should they build first? Registration? Login? Password reset?

They don't know.

But if you give them this User Story:

> “As a new student, I want to register so that I can create an account.”

Now they know the specific feature they need to build.

> **The advantage:** the team can understand, develop, test, and track each feature separately.

4 What happens if we don't create Epics and User Stories?

Imagine the customer gives these 10 requirements:

- Student registration
- Student login
- Password reset
- Update profile
- Upload profile photo
- View marks
- Download marksheet
- View attendance
- Receive notifications
- Change password

Without organizing them, the team sees one long list of 10 requirements.

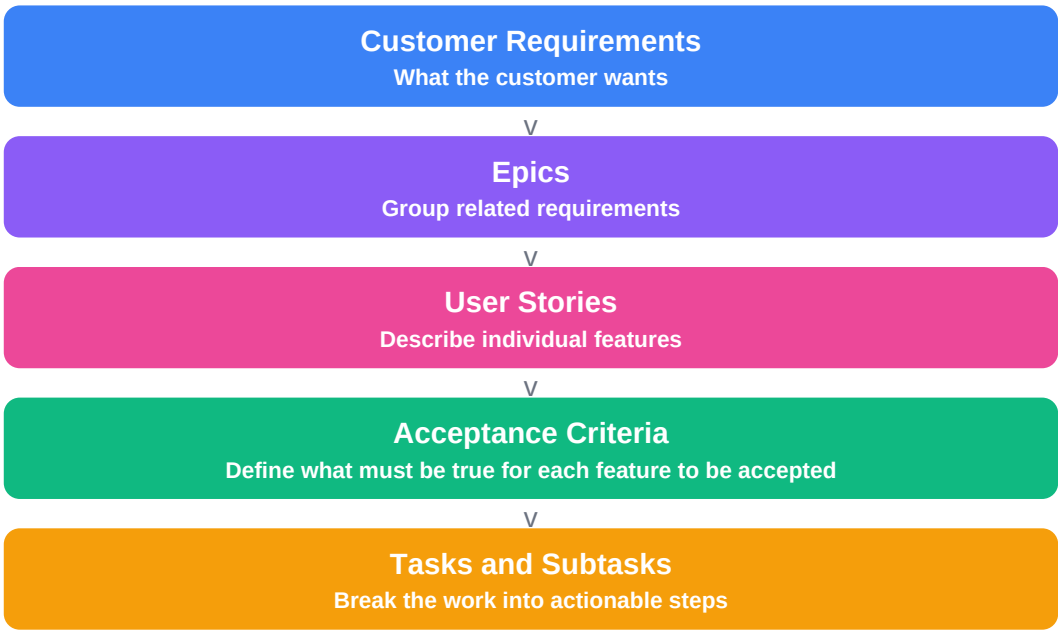
With Epics and User Stories, the list becomes organized:

Epic	User Stories
Account Management	Registration, Login, Password Reset, Change Password
Profile Management	Update Profile, Upload Photo
Academic Management	View Marks, Download Marksheet, View Attendance
Notifications	Receive Notifications

Now the team can easily see which features belong together.

> **Important:** We don't create Epics and User Stories because the software cannot be built without them. We create them to make the work easier to understand, organize, prioritize, assign, and track.

5 Let's understand the complete flow



6 Check your understanding

Imagine the customer says:

> “I want students to register, log in, and reset their passwords.”

Which option correctly explains an Epic and User Stories?

Choose your answer:

A. Registration is the Epic, and the entire college app is one User Story.

B. Account Management is the Epic, and Registration, Login, and Password Reset are separate User Stories.

C. We don't need to organize requirements; the developer can guess what to build.